

Pandas – Базовый уровень

Pandas

Series

Зачастую для анализа используются данные, выгруженные из базы в виде файлов, например, в формате CSV. Работа с выгруженными данными позволяет производить более глубокую аналитику и проверять большее количество гипотез. Для анализа данных можно использовать различные табличные редакторы. Однако табличные редакторы не всегда обладают достаточным функционалом для глубокой аналитики и не поддерживают обработку больших массивов данных. Поэтому на практике для анализа чаще используют возможности языка Python и, в частности, **библиотеки pandas**.

Работа с библиотекой pandas начинается с импорта. Общепринятое сокращение для библиотеки pandas — `pd`.

```
import pandas as pd
```

Pandas рассматривает таблицу как набор колонок, а саму колонку — как список, в котором последовательно представлены все значения колонки. Для проведения глубокого анализа данных не хватает набора действий, которые можно совершить с обычным списком. Например, можно выбрать одно или несколько значений из списка или добавить в него новые элементы. Поэтому для представления данных колонок используется специальный тип данных — **Series**. Чтобы создать из обычного списка Series, нужно воспользоваться конструктором данных Series и передать в него список значений в квадратных скобках:

```
col = pd.Series([18, 15, 13, 11, 13, 5])
```

Арифметические операции

Помимо операций, которые можно производить с обычным списком, Series обладает рядом дополнительных операций, полезных при проведении анализа данных. Например, можно использовать арифметические операции (сложение, вычитание, умножение, деление) как между двумя Series, так и между Series и константой.

Арифметические операции, например сложение, между двумя Series производятся поэлементно, то есть первое значение первого Series складывается с первым значением второго Series, второе значение — со вторым и т. д. Важно, что количество значений в двух Series должно быть одинаковым.

```
col1          # [18, 15, 13, 11, 13,  5]
col2          # [18, 15, 13, 11, 13,  5]
col1 + col 2  # [36, 30, 26, 22, 26, 10]
```

При сложении Series с константой pandas представляет константу как Series, состоящий из нужного количества элементов, каждый из которых равен константе. Этот процесс называется Broadcasting. Далее вычисление происходит также поэлементно.

Broadcasting — автоматическое преобразование, которое позволяет представить объект в пространстве с большим количеством измерений.

```
col          # [18, 15, 13, 11, 13,  5]
col + 3      # [21, 18, 16, 14, 16,  8]
```

–

Операции сравнения

Тип данных Series поддерживает операции сравнения, которые также применяются поэлементно. В результате получается новый Series, состоящий из значений True и False. Значения True будут находиться на позиции тех элементов исходного Series, которые соответствуют условию сравнения, а значения False — на позициях тех элементов, которые условию не соответствуют.

```
col          # [ 18, 15, 13, 11, 13, 5]
col = 13      # [False, False, True, False, True, False]
col > 11      # [ True, True, True, False, True, False]
```

Такие Series называются **фильтрами**. Фильтр можно передать в квадратных скобках в исходный Series. В результате получится новый Series, в котором останутся только те значения, которые соответствуют значениям True в фильтре.

```
col          # [ 18, 15, 13, 11, 13, 5]
col > 11      # [ True, True, True, False, True, False]
col[col > 11] # [ 18, 15, 13,          13      ]
```

Логические операции

Несколько фильтров можно объединить, используя логические операторы (& — И, | — ИЛИ, ~ — НЕ). Важно обособлять каждый фильтр круглыми скобками и учитывать приоритет операций.

```
col          #[18, 15, 13, 11, 13, 5]
col[(col > 11) & (col < 15)] # [13, 13]
col[(col < 11) | (col > 15)] # [18, 5]
```

Методы

Для работы с Series существует ряд методов:

- `sum` — сумма значений,
- `mean` — среднее арифметическое,
- `median` — медиана,
- `max/min` — максимум/минимум,
- `unique` — все уникальные значения,
- `nunique` — число уникальных значений.

Таким образом, тип данных Series представляет собой усовершенствованный список. Этот тип данных поддерживает арифметические операции, операции сравнения и логические операции, обладает рядом дополнительных методов, полезных для анализа данных. Поэтому при работе с колонками таблиц в pandas используется тип данных Series.

Dataframe

Для представления таблиц в pandas есть специальный тип данных — **DataFrame**. Если тип данных Series можно воспринимать как усовершенствованный список, то DataFrame — это усовершенствованный словарь.

Словарь

Словарь используют для сопоставления одних данных другим. Он состоит из ключей и значений. Ключи уникальны и используются для идентификации данных. В созданный словарь можно последовательно добавлять пары «ключ — значение».

```
alphabet = {}  
alphabet['A'] = 1  
alphabet['B'] = 2
```

Кроме того, можно добавить пары «ключ-значение» непосредственно при создании словаря.

```
alphabet = {'A': 1, 'B': 2}
```

Ключ позволяет выбрать значение из словаря. Для этого нужно указать интересующий ключ в квадратных скобках.

```
alphabet = {'A': 1, 'B': 2}  
alphabet['B'] # 2
```

Ключом может быть любой неизменяемый тип данных. А значения словаря могут быть представлены любым типом данных, в том числе списком. Поэтому таблицу можно представить как словарь, в котором ключом является название столбца. Ключ в таком словаре сопоставляется со списком значений соответствующей колонки.

```
user = {  
    'id': [1, 2, 3, 4],  
    'birthday': ['13.02.2012', '27.12.2008', '11.08.2005', '01.10.2009'],  
    'gender': ['М', 'Ж', 'М', 'Ж']  
}
```

Создание DataFrame

Чтобы создать DataFrame на основе словаря достаточно передать словарь в конструктор данных DataFrame.

```
user = pd.DataFrame({
    'id': [1, 2, 3, 4],
    'birthday': ['13.02.2012', '27.12.2008', '11.08.2005', '01.10.2009'],
    'gender': ['М' , 'Ж' , 'М' , 'Ж']
})
```

Индексация

DataFrame позволяет получить набор названий колонок или посчитать количество строк.

```
user.columns # [id, birthday, gender]
len(users) # 4
```

Из DataFrame можно получить список значений колонки, передав её название в качестве ключа в квадратных скобках. При этом результат будет представлен типом данных Series. Также, можно получить значения сразу нескольких колонок, указав их названия списком в квадратных скобках. В этом случае в результате получится новый DataFrame.

```
user['id']
user[['id', 'birthday']]
```

Чтобы получить строку таблицы, можно использовать свойство `iloc`, указав индекс строки в квадратных скобках. Индексация строк начинается с 0. В результате также получится Series, из которого можно забрать значения, указав индекс или название колонки.

```
row = user.iloc[2]

row[0] # 3
row[1] # 11.08.2005

row['id'] # 3
row['birthday'] # 11.08.2005
```

Фильтрация

К DataFrame можно применять такие же фильтры, как и к Series. При этом в результате получится новый DataFrame, содержащий только те строки исходной таблицы, которые удовлетворяют условию фильтрации.

```
user[user['gender'] == 'Ж']
user[(user['gender'] == 'Ж') | (user['id'] == 3)]
```

Итерация по строкам в цикле

Чтобы совершить какие-либо действия со строками, можно использовать итерацию в цикле. Для этого можно использовать функцию `iterrows`. При этом в каждой итерации будет возвращаться номер строки и сама строка в виде `Series`.

```
for row_ind, row in df.iterrows():  
    print(row['gender'])
```

Функция `iterrows` работает очень медленно, поэтому на практике для итерации по строкам используют функцию `itertuples`, которая предоставляет строки как именованные кортежи (`namedtuple`) и работает на порядок быстрее. В цикле можно указать название колонки, которую, например, нужно вывести для каждого именованного кортежа (`tup`). Важно, что так можно сделать только с колонками с латинскими названиями без пробелов.

```
for tup in df.itertuples():  
    print(tup.gender)
```

Операции с таблицами

В DataFrame можно вносить изменения, например, добавить новые колонки, изменить значения колонок, удалять или переименовывать колонки. Для добавления новой колонки достаточно указать её название в квадратный скобках и передать Series со значениями колонки. Важно, чтобы количество значений совпадало с количеством строк в DataFrame. Также можно передать не Series, а просто список значений: DataFrame автоматически преобразует этот список в Series при добавлении колонки.

```
user['grade'] = \
    pd.Series([5, 9, 11, 8])
```

```
user['grade'] = [5, 9, 11, 8]
```

Для изменения значений колонки нужно указать её название в квадратных скобках и передать список новых значений в правильном порядке.

```
user['gender'] = \
    ['Мужской' , 'Женский' , 'Мужской' , 'Женский']
```

Для удаления колонки используется функция `drop`. После ключевого слова `columns` указывается название или список названий удаляемых колонок.

```
user.drop(columns=['grade'])
```

Важно понимать, что в результате применения этой функции получается новый DataFrame. Если нужно, чтобы изменения произошли в исходном DataFrame, нужно присвоить переменной новое значение.

```
user = \
    user.drop(columns=['grade'])
```

Для изменения названий колонок используется функция `rename`, которая также возвращает новый DataFrame. После ключевого слова `columns` нужно указать словарь переименований, то есть пары старых и новых названий колонок.

```
user = \
    user.rename(columns={
        'id': 'user_id'
    })
```

Таким образом, табличные данные в pandas представлены в виде типа данных DataFrame, который можно воспринимать как усовершенствованный словарь. В DataFrame можно использовать расширенную индексацию, фильтрацию, совершать операции над строками и колонками, что необходимо для анализа данных.

Чтение данных

Для выгрузки из баз данных чаще всего используют файлы в формате CSV, для чтения которых в pandas используют функцию `read_csv`. Также существуют функции для чтения других форматов, например `read_excel` для чтения файлов в формате xls или xlsx. В функцию `read_csv` необходимо передать путь к файлу.

```
user = pd.read_csv('user.csv')
```

В файлах формата CSV вся информация представлена как текст, разделённый каким-либо символом. По умолчанию pandas считает, что разделителями значений в файле является запятая. Если в файле используется иной разделитель, это нужно явно показать, прописав разделитель после параметра `sep`.

```
user = pd.read_csv('user.csv', sep=';')
```


Определение типов данных

В файле формата CSV не содержится информации о типах данных. Pandas автоматически распознает типы данных, но не всегда делает это корректно. Чтобы проверить тип данных для колонки, нужно использовать функцию `dtype`.

```
user['id'].dtype      # int
user['birthday'].dtype # string
user['gender'].dtype  # string
user['grade'].dtype   # float
```

К неправильному автоматическому определению типа данных может привести несколько причин. Например, колонка с моментами времени может быть распознана как строковая, если в ней находятся даты в нестандартном для pandas формате. А наличие пустых значений в колонке с числами приводит к тому, что pandas распознаёт её как тип `float`. Поэтому не стоит полагаться на автоматическое определение типов в pandas. Стоит проверять, насколько верно были определены типы, и при необходимости определять типы самостоятельно.

Чтобы принудительно указать, что в колонке хранятся моменты времени, используют функцию `to_datetime`. Она принимает колонку со строковыми значениями и возвращает колонку со значениями типа «момент времени». В функцию нужно передать название колонки и указать формат даты, в котором хранятся данные в исходном файле.

Формат прописывают при помощи шаблона, элементы которого показывают позицию и вид отображения года, месяца, дня и так далее. Например, `%d` — день, `%m` — месяц, `%Y` — год, `%H` — час, `%M` — минута, `%S` — секунда. Подробнее с кодами форматов для типа «момент времени» можно ознакомиться на соответствующей странице [документации языка программирования Python](#).

```
user = pd.read_csv('user.csv')
user['birthday'] = pd.to_datetime(user['birthday'], format='%d.%m.%Y')
```

Интересно, что функцию `to_datetime` можно использовать для преобразования не только целой колонки, но и для отдельного строкового значения. Это свойство может быть полезным при создании фильтра по дате. В аргументах функции можно написать интересующую дату и указать её формат, а функция `to_datetime` преобразует эту дату в момент времени, с которым можно сравнивать значения колонки с датами.

```
user['birthday'] >= pd.to_datetime('01.01.2009', format='%d.%m.%Y')
```

Чтобы понять, есть ли в колонке пустые значения, можно использовать функцию `isna` — она возвращает Series из значений True и False. Значение True в таком Series находится на позициях пустых значений исходной колонки. Логические значения можно суммировать при помощи функции `sum`. При сложении значение True будет интерпретироваться как 1, а значение False — как 0. Если получившаяся сумма отличается от 0, значит, в колонке присутствуют пустые значения.

```
user['grade'].isna().sum()
```

В некоторых случаях пустые значения заменяют на 0. Однако иногда такой подход искажает данные. Например, в колонке с информацией о классе, в котором учится пользователь, лучше заменить пустые значения не на 0, а на среднее значение. Чтобы посчитать среднее, нужно сначала убрать все пустые значения при помощи функции `dropna`. Для расчёта среднего значения нужно использовать функцию `mean` и округлить результат до целого числа функцией `round`.

```
mean_grade = user['grade'].dropna().mean().round()
```

Теперь можно использовать рассчитанное среднее для замены пустых значений. Пустые значения можно заменить при помощи функции `fillna`, в которую нужно передать значение, на которое нужно заменить пустые значения. Также стоит добавить функцию `astype` со значением `'int'`, чтобы определить для колонки целочисленный тип данных.

```
user['grade'] = \
    user['grade'].fillna(mean_grade).astype('int')
```

Полностью чтение файла и определение типов данных будет выглядеть следующим образом:

```
user = pd.read_csv('user.csv')
```

```
user['id'] = user['id'].astype('int')
```

```
user['birthday'] = pd.to_datetime(user['birthday'],
                                   format='%d.%m.%Y')
```

```
user['gender'] = user['gender'].astype('str')
```

```
mean_grade = user['grade'].dropna().mean().round()
user['grade'] = \
    user['grade'].fillna(mean_grade).astype('int')
```

Специальные типы данных

Для некоторых типов данных в pandas представлен дополнительный инструментарий, к которому можно получить доступ, указав соответствующий атрибут. Для колонки, которая полностью состоит из данных типа «момент времени», это атрибут `dt`. После этого атрибута нужно указать название функции или свойства. Например, можно преобразовать моменты времени, оставив только годы или любую другую характеристику момента времени: месяц, день, часы, минуты, секунды.

```
user['birthday'].dt.year
user['birthday'].dt.month
user['birthday'].dt.day
user['birthday'].dt.hour
user['birthday'].dt.minute
user['birthday'].dt.second
```

Для работы с датами полезны функции `round`, `floor` и `ceil`. Они позволяют округлить моменты времени, чтобы избавиться от лишней детализации, например секунд или минут.

Функция `round` округлит по правилам математики, `floor` — всегда в меньшую сторону, `ceil` — всегда в большую. В аргументах этих функций нужно указать, как следует округлять: `'D'` — до дней, `'H'` — до часов, `'T'` — до минут, `'S'` — до секунд.

```
user['birthday'].dt.round('D')
user['birthday'].dt.floor('H')
user['birthday'].dt.ceil('T')
```

Для округления момента времени вниз до месяца используется иной подход: сочетание функций `to_period` с аргументом `'M'` и `to_timestamp`. Важно, что атрибут `dt` нужно указывать при вызове каждой функции для работы с моментом времени.

```
user['birthday'] \
    .dt.to_period('M') \
    .dt.to_timestamp()
```

С моментами времени можно совершать арифметические действия. Например, по дате рождения можно вычислить возраст по состоянию на текущий момент времени. Для этого нужно вычесть из текущего момента времени дату рождения. В результате такого вычисления получится `Series`, содержащий разницы времён. Специальная функция `total_seconds` позволит выразить эту разницу в количестве секунд. Полученный результат можно использовать для фильтрации по возрасту, переводя интересующее возрастное ограничение в секунды.

```
current_date = pd.to_datetime('01.09.2023', format='%d.%m.%Y')
diff_to_current = current_date - user['birthday']
diff_to_current.dt.total_seconds() < 60 * 60 * 24 * 364 * 10
```

Специальные свойства есть не только у данных типа «момент времени», но и у колонок, целиком состоящих из строковых данных. Доступ к дополнительному функционалу можно получить, используя атрибут `str`. Например, можно заменить некоторые значения колонки, используя функцию `replace`, в которую нужно передать старое и новое значения.

```
user['gender'] \
    .str.replace('М', 'Мужской') \
    .str.replace('Ж', 'Женский')
```

Также можно воспользоваться привычными функциями для работы со строками: `lower`, `upper`, `contains` и др.

Соединение таблиц

Для соединения нескольких таблиц используется функция `merge`. Та таблица, для которой вызвана эта функция, становится условно «левой». В аргументах функции нужно указать, с какой таблицей следует произвести соединение, то есть условно «правую». Также в аргументах `left_on` и `right_on` нужно указать названия колонок в соответствующих таблицах, по которым будет произведено соединение. Если название этих колонок в обеих таблицах одинаковое, допускается прописать его один раз в аргументе `on`. Можно соединять таблицы по нескольким колонкам сразу, указав их названия в виде списков.

```
course_age = user_course \
    .merge(user, left_on='user_id', right_on='id')
```

В результате соединения двух таблиц получается новая таблица, которую можно соединить с ещё одной таблицей.

```
course_age = user_course \
    .merge(user, left_on='user_id', right_on='id') \
    .merge(course, left_on='course_id', right_on='id')
```

Может оказаться, что названия колонок в исходных таблицах были одинаковыми. В этом случае после соединения к названиям колонок по умолчанию будут добавлены суффиксы: `_x` — для колонки из левой таблицы и `_y` — для колонки из правой таблицы. Можно указать суффиксы самостоятельно, передав их список в аргументе `suffixes`.

```
course_age = user_course \
    .merge(user, left_on='user_id', right_on='id') \
    .merge(course, left_on='course_id', right_on='id', suffixes=('_x', '_y'))
```

По умолчанию функция `merge` производит внутреннее соединение таблиц. В аргументе `how` можно указать другой тип соединения, например `'left'` (левое соединение) или `'right'` (правое соединение).

```
course_age = user_course \
    .merge(user, left_on='user_id', right_on='id') \
    .merge(course, left_on='course_id', right_on='id', how='inner')
```

Группировка и агрегация

Чтобы сгруппировать данные по какому-либо столбцу, нужно использовать функцию `groupby`. В аргументах функции нужно указать колонку или список колонок, по которым производится группировка. Во избежание непредвиденного результата в аргументах этой функции следует указать `as_index=False`.

```
subject_mean_age = course_age \
    .groupby('subject', as_index=False)
```

Для агрегации данных используется функция `agg`. В качестве её аргументов нужно указать словарь колонок и соответствующих им функций агрегации. При агрегации можно посчитать среднее значение ('mean'), максимальное ('max') и минимальное ('min'), количество значений ('count') или количество уникальных значений ('nunique').

```
subject_mean_age = course_age \
    .groupby('subject', as_index=False) \
    .agg({'age': 'mean'})
```

Помимо встроенных функций агрегации можно использовать пользовательскую функцию. На вход такая функция будет принимать Series, а возвращать какое-то посчитанное значение.

```
def agg_f(input):
    return input.max() - input.min()

subject_mean_age = course_age \
    .groupby('subject', as_index=False) \
    .agg({'age': agg_f})
```

Простые функции можно превратить в лямбда-функции и прописать непосредственно внутри аргумента функции `agg`.

```
subject_mean_age = course_age \
    .groupby('subject', as_index=False) \
    .agg({'age': lambda x: x.max() - x.min()})
```

Сортировка

Для сортировки значений используют функцию `sort_values`. В качестве аргумента в этой функции нужно указать колонку или список колонок, по которым нужно сортировать данные.

По умолчанию строки сортируются по возрастанию значений. Чтобы сортировать данные по убыванию значений, в аргументе `ascending` нужно указать значение `False`.

```
subject_mean_age.sort_values('age', ascending=False)
```